Fraud Detection Pipeline

Technical Report

Data Science Take-Home Assessment

March 2026

Contents

1. Executive Summary

This report documents an end-to-end fraud detection pipeline built for the Fraudio data science take-home assessment. The pipeline loads transactional payment data, performs data quality checks, engineers fraud-relevant features, trains four classifiers, evaluates them with a focus on precision-recall trade-offs, and optimises for target recall levels between 30% and 60%.

Key results:

- Dataset: 157,306 training transactions (0.11% fraud rate after label extraction), 110,998 test transactions
- After dedup

2. Data Quality Assessment

2.1 Raw Data Overview

Property	Train	Test
Rows (raw)	157,306	110,998
Columns	48 (incl. 4 labels)	44
Fraud rate	0.11%	Unknown
Date range	Feb 2020+	Apr 2020+

The test set is a temporal hold-out starting April 2020, with 4 label columns (cb_fraudflag, chargeback_bank_date, chargeback_reason_code, fraudimportdate) removed -- consistent with a real-world deployment scenario.

2.2 Quality Pipeline (6 Steps)

The data quality pipeline applies the following transformations in order:

Step 1: Null Normalisation

Sentinel strings ("none", "n/a", "na", "nan", "null", "") are replaced with np.nan across all object columns after lowercasing and stripping whitespace. This ensures consistent missing-value handling downstream.

Step 2: Deduplication

Duplicate transactionid rows are removed. The deduplication strategy prioritises: (1) fraud-positive rows first (so chargebacks are never silently discarded), then (2) transaction-type priority (refund > auth_capture > capture > auth). This reduced train from 157,306 to 113,905 rows and test from 110,998 to 74,792 rows.

Step 3: Response Code Encoding Fixes

Garbled UTF-8 encoded response codes (e.g. "limitgjberschritten,doch-funktionmg6glich") are mapped to clean labels (e.g. "limit_exceeded_possible").

Step 4: Type Casting

Boolean string columns (cvvused, recurring, threedsused, etc.) are cast to Int8. Numeric fields (cardbin, euramount) are coerced. ISO 8601 timestamps are parsed to datetime with UTC awareness.

Step 5: Drop Identifiers

High-cardinality identifiers with no predictive signal are dropped: transactionid, transactionip, approval_code, submerchant, merchanturl. Note that saltedhash is retained at this stage because it is needed for velocity feature engineering.

Step 6: Quality Report

A comprehensive quality report is logged showing missing-value percentages and cardinality per column. Key findings from the cleaned training set:

Column	Missing %	Notes
merchantcountry	100.0%	Entirely null -- dropped in FE

cardholder_disposabledomain	99.0%	Nearly all null
decline_type	57.6%	Missing when txn approved
State	56.3%	Geographic data gap
City	22.8%	Geographic data gap
geoip_country_code	4.7%	Minor gaps
issuingbank	2.0%	Minor gaps

3. Feature Engineering

Feature engineering is applied after the temporal train/validation split to prevent data leakage. Lookup tables are built from the training fold only -- validation and test rows never influence training statistics.

3.1 Timestamp Features

From authtimestamp: hour (0-23), dayofweek (0-6), is_weekend (binary), day (1-31). These capture temporal fraud patterns (e.g. higher fraud rates during certain hours).

3.2 Country Mismatch Flags

Two binary flags engineered to detect geographic inconsistencies: card_billing_mismatch (card issuing country != billing country) and geoip_billing_mismatch (IP geolocation country != billing country). Cross-border mismatches are strong fraud signals.

3.3 Velocity & Aggregation Features

card_txn_7d: Rolling 7-day transaction count per card, computed via binary search on sorted timestamp history. This captures burst-like fraud patterns without lifetime bias.

card_avg_amount: Lifetime average transaction amount per card. amount_deviation: Current transaction amount minus card average -- flags abnormally large or small transactions for a given cardholder.

merchant_txn_count: Total transaction count per merchant -- proxies merchant popularity / legitimacy.

3.4 Domain Features

domain_freq: Frequency encoding of cardholder email domain. Rare or disposable domains may indicate synthetic identities.

3.5 Amount Features

log_euramount: Log1p transformation of euramount to reduce skewness and improve model performance on right-skewed transaction amounts.

3.6 Final Feature Set (27 features)

Type	Count	Features
Numeric	20	euramount, log_euramount, card_txn_7d, card_avg_amount, merchant_txn_count, amount_deviation, hour, dayofweek, day, is_v
Categorical	6	cardbrand, cardtype, transactiontype, channel, terminaltype, currencyname, brandcardty

Preprocessing: numeric features are median-imputed and standard-scaled. Categorical features are constant-imputed ('missing') and one-hot encoded.

4. Class Imbalance Handling

The dataset exhibits extreme class imbalance with a fraud rate of approximately 0.11-0.16% (36 fraud cases in the validation set of 22,781 rows). This ratio of ~630:1 makes standard accuracy meaningless and requires specialised strategies.

4.1 Strategies Employed

- Logistic Regression & Random Forest: class_weight='balanced' -- automatically adjust weights inversely proportional to class frequencies.
- XGBoost: s
  the training

4.2 Evaluation Metric Choice

Given the extreme imbalance, we prioritise PR-AUC (Average Precision) over ROC-AUC as the primary selection metric. PR-AUC is more informative when the positive class is rare because it focuses on the precision-recall trade-off rather than the true-negative-dominated ROC space.

5. Model Training & Architecture

5.1 Temporal Train/Validation Split

A temporal split (80/20) is used instead of random stratified splitting. Data is sorted by authtimestamp; the earliest 80% forms the training set and the most recent 20% forms validation. This mirrors production deployment where models predict on future transactions. Train: 91,124 rows, Val: 22,781 rows (0.16% fraud).

5.2 Model Specifications

Logistic Regression

```
class_weight='balanced', max_iter=1000, random_state=42
Preprocessor: StandardScaler + OneHotEncoder
```

Random Forest

```
n_estimators=300, max_depth=15, min_samples_leaf=20
class_weight='balanced', n_jobs=-1, random_state=42
```

XGBoost

```
n_estimators=500, max_depth=6, learning_rate=0.05
scale_pos_weight=auto, subsample=0.8, colsample_bytree=0.8
eval_metric='aucpr', random_state=42
```

LightGBM

```
n_estimators=500, max_depth=6, learning_rate=0.05
is_unbalance=True, subsample=0.8, colsample_bytree=0.8
random_state=42, verbose=-1
```

6. Evaluation Metrics & Overfitting Analysis

6.1 Validation Metrics Summary

Model	ROC-AUC	PR-AUC	F1	Precision	Recall
LogisticRegression	0.9315	0.0216	0.013	0.01	0.92
RandomForest	0.9291	0.0115	0.021	0.01	0.75
XGBoost	0.9320	0.0508	0.071	0.04	0.25
LightGBM	0.8118	0.0065	0.016	0.01	0.78

XGBoost achieves the highest PR-AUC (0.051) -- the most relevant metric given the extreme imbalance. While Logistic Regression achieves the highest recall (0.92), it does so at very low precision (0.01), producing excessive false positives.

6.2 Overfitting Analysis

Model	Train ROC	Val ROC	Train PR	Val PR	Gap PR
LogisticRegression	0.9491	0.9315	0.0797	0.0216	0.058
RandomForest	0.9780	0.9291	0.1733	0.0115	0.162
XGBoost	0.9997	0.9320	0.8621	0.0508	0.811
LightGBM	0.8587	0.8118	0.0075	0.0065	0.001

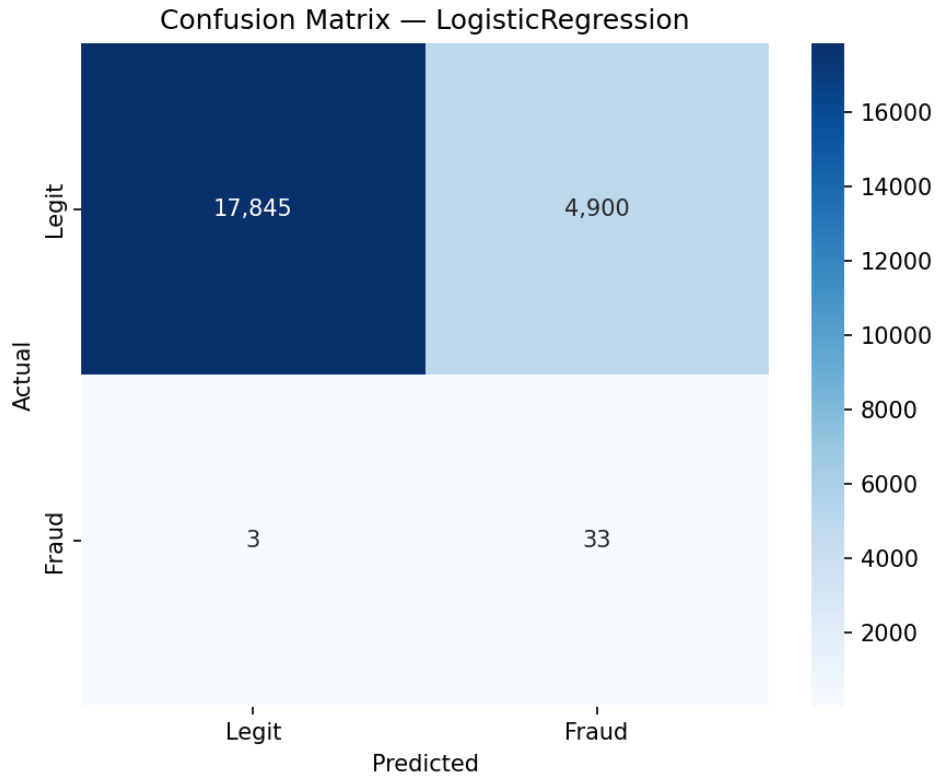
XGBoost shows significant overfitting (train PR-AUC 0.86 vs val 0.05), typical of gradient boosting on highly imbalanced data with 500 estimators. However, it still achieves the best validation PR-AUC. Potential mitigations:

- Reduce n_estimators or add early stopping on validation PR-AUC
- Increase re

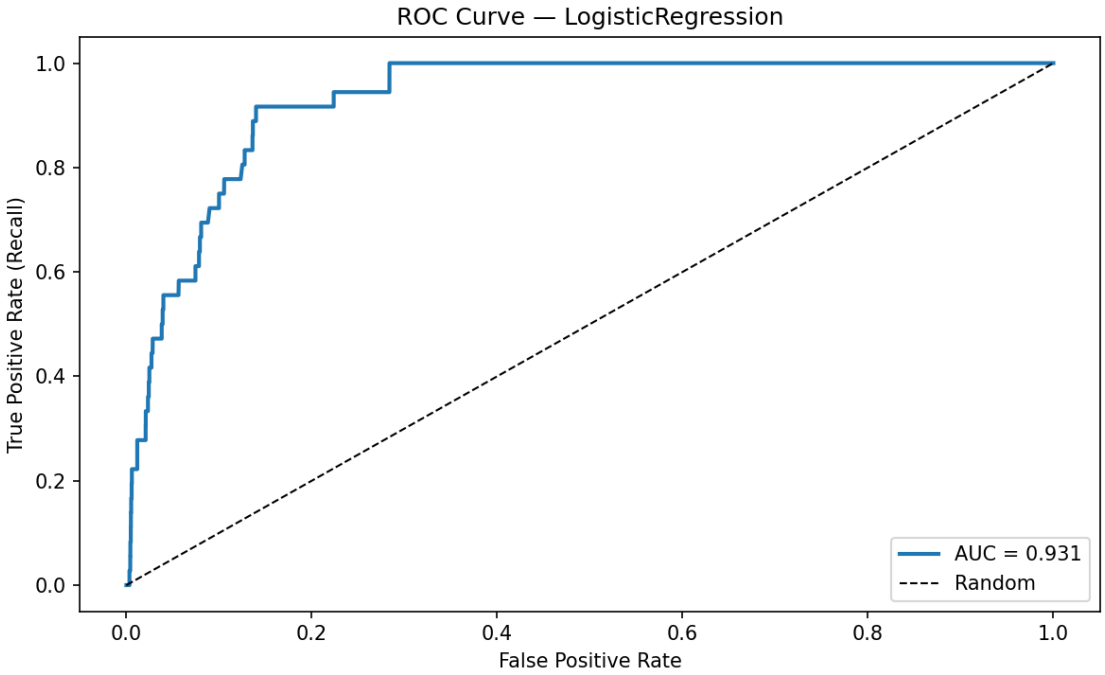
LightGBM shows the smallest train/val gap (0.001), but its absolute performance is the weakest. Logistic Regression is a reasonable middle ground with moderate overfitting and competitive ROC-AUC.

6.3 LogisticRegression -- Detailed Plots

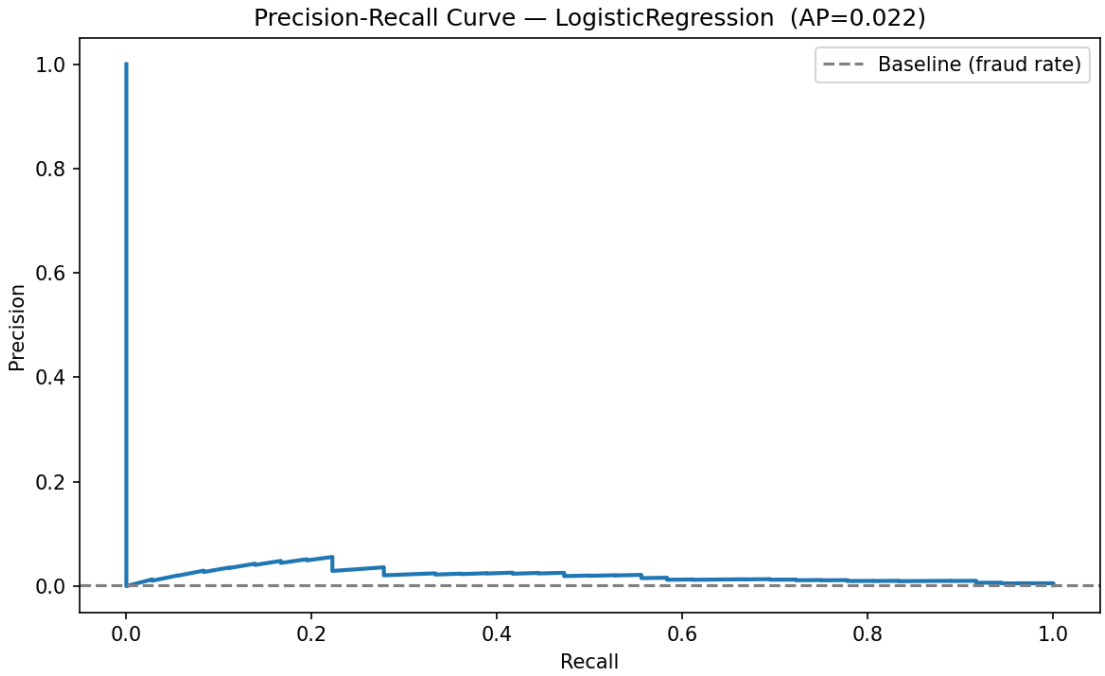
Confusion Matrix



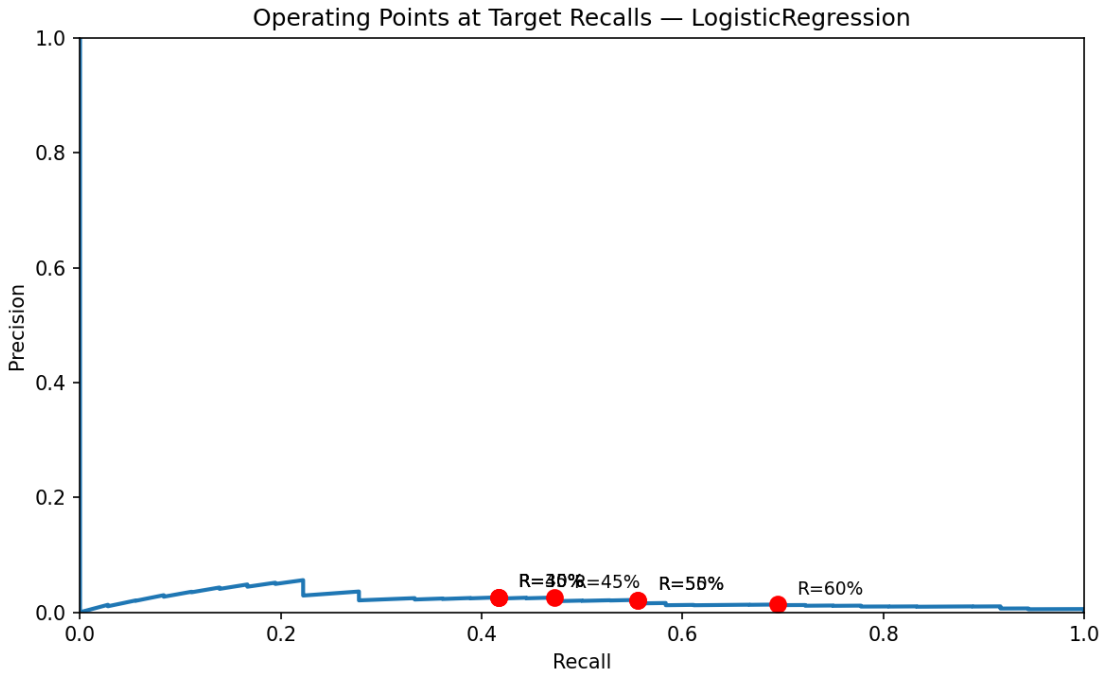
ROC Curve



Precision-Recall Curve

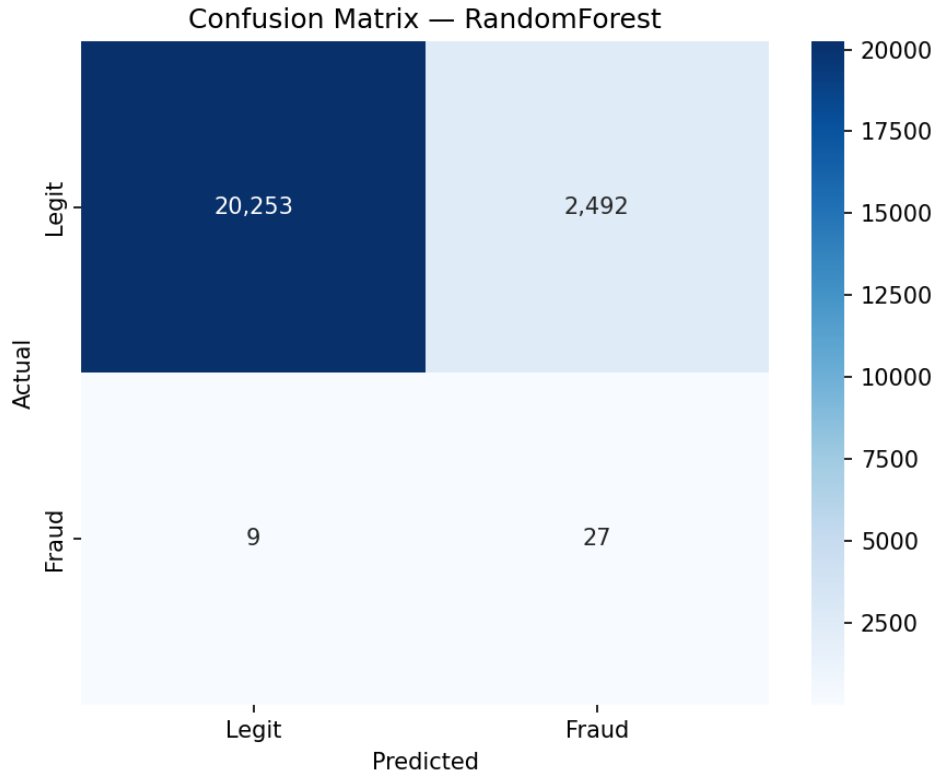


Recall Operating Points

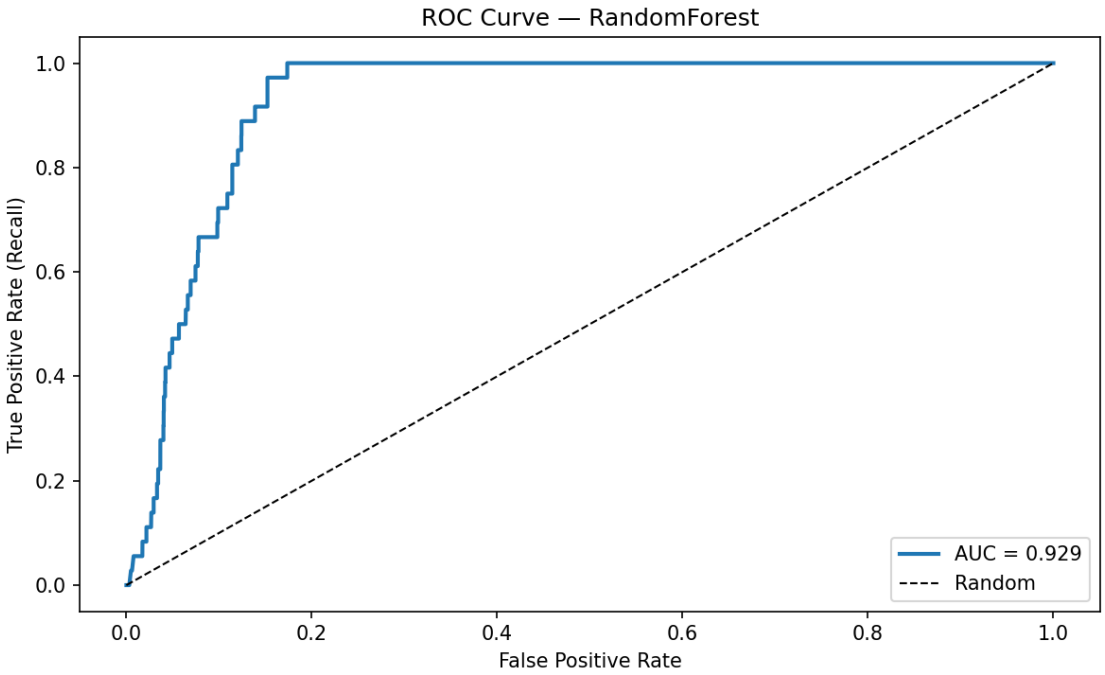


6.3 RandomForest -- Detailed Plots

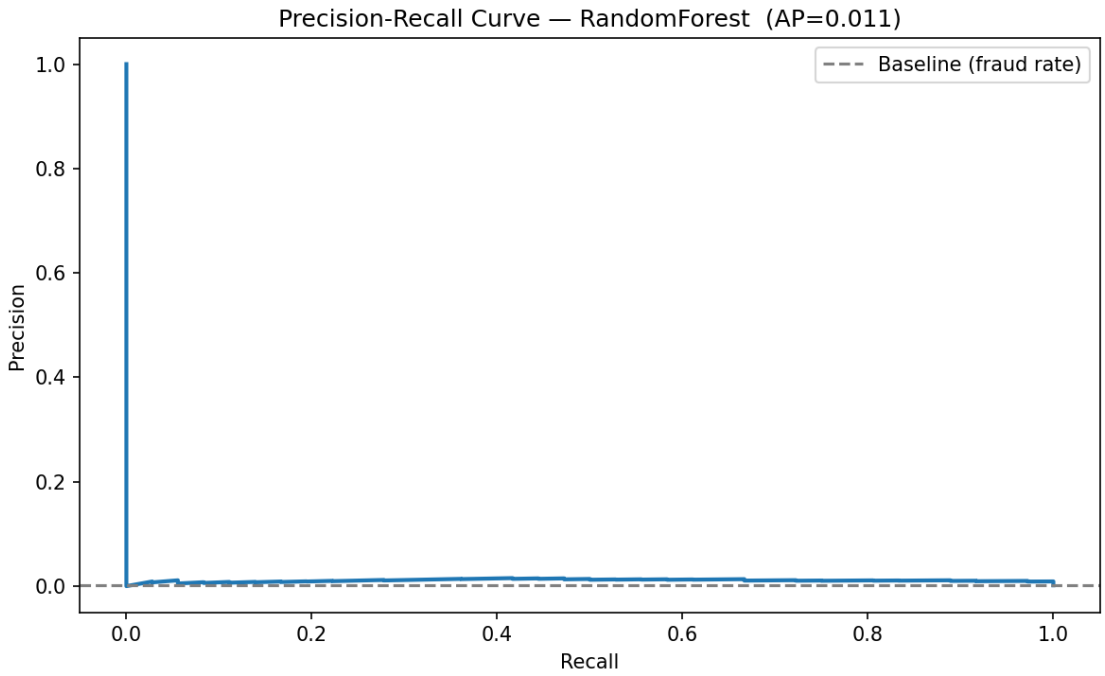
Confusion Matrix



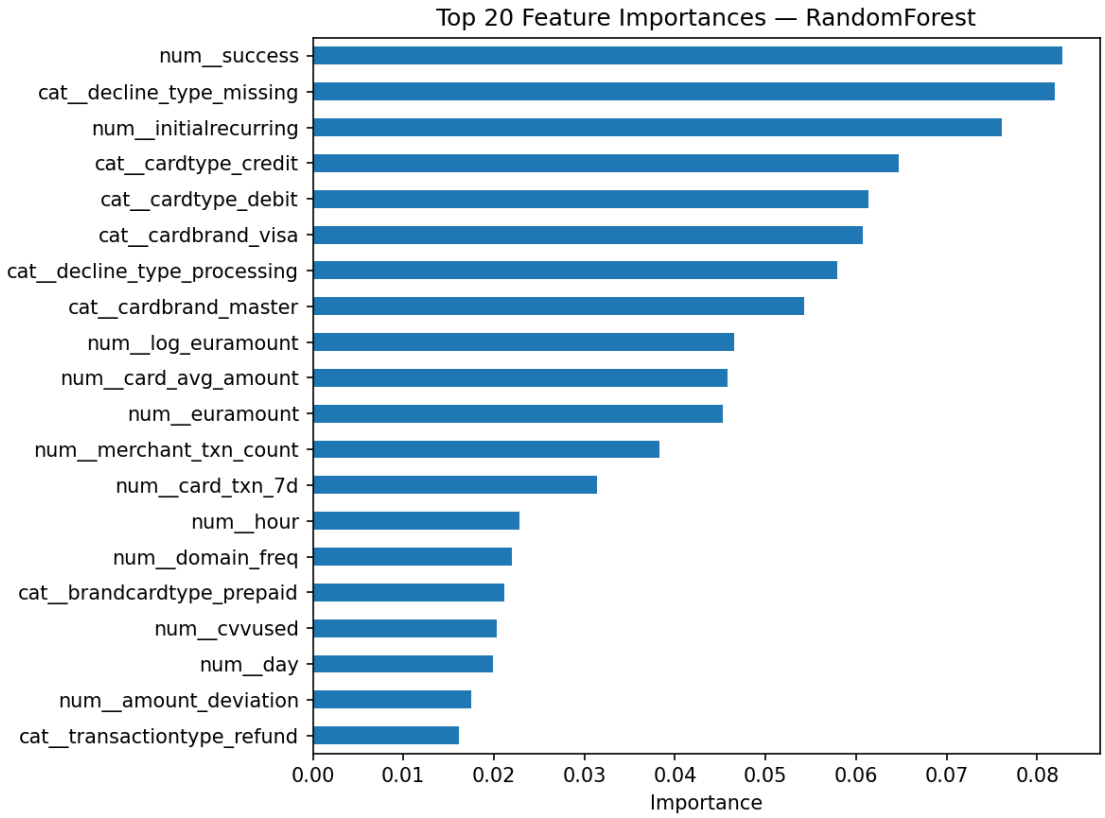
ROC Curve



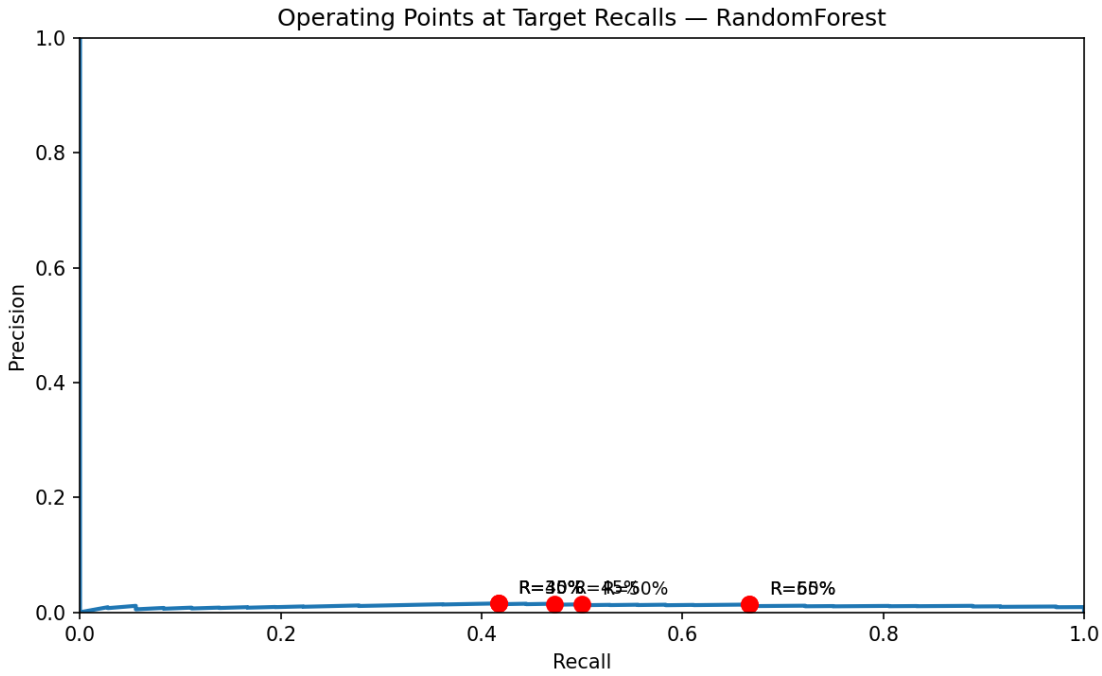
Precision-Recall Curve



Feature Importance

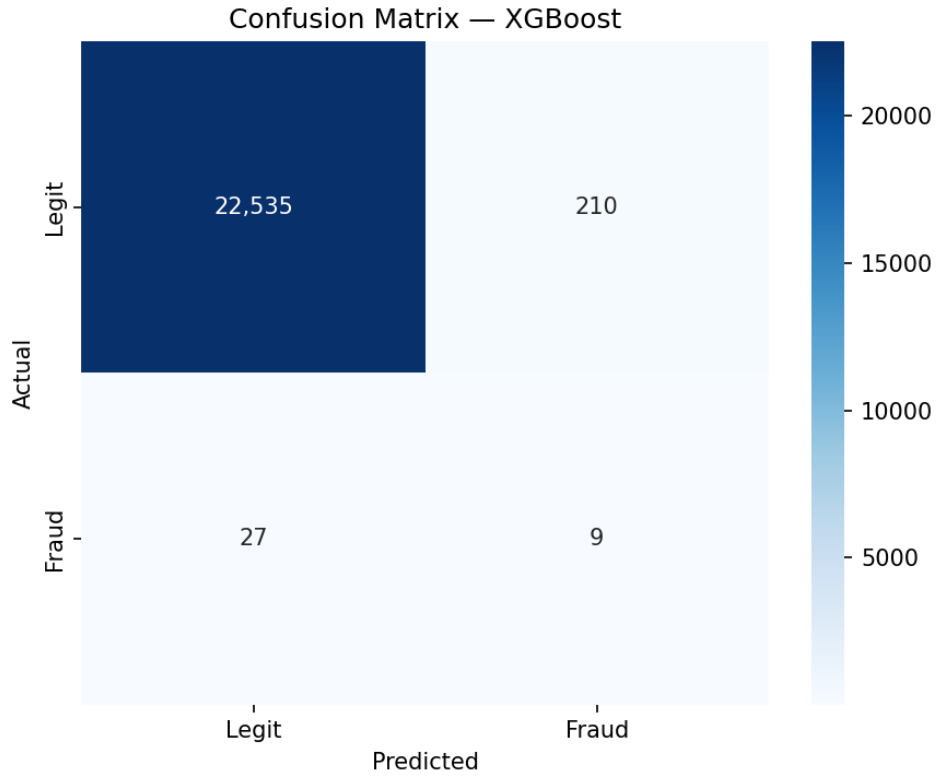


Recall Operating Points

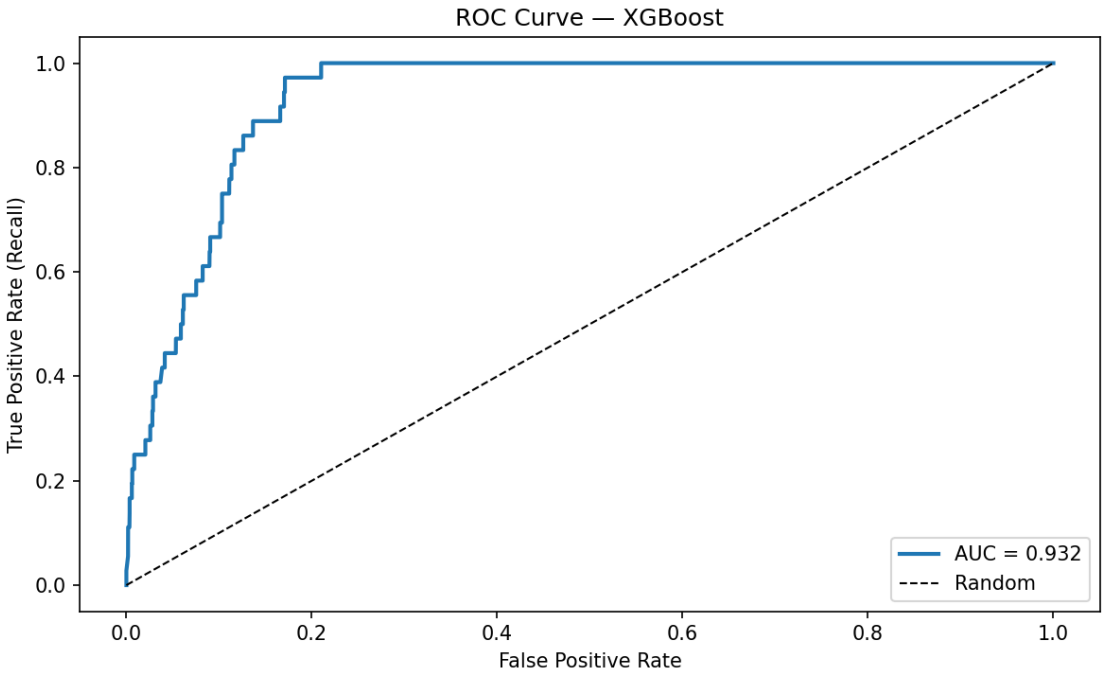

Operating Points at Target Recalls — RandomForest
Precision
Recall
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
R=60%

6.3 XGBoost -- Detailed Plots

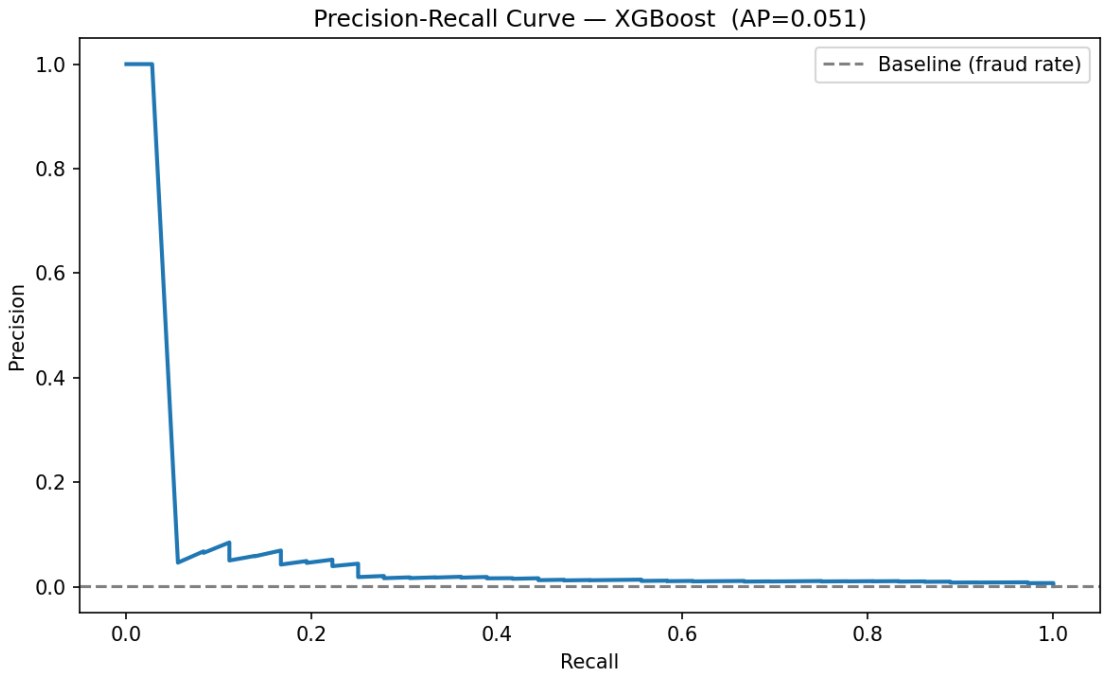
Confusion Matrix



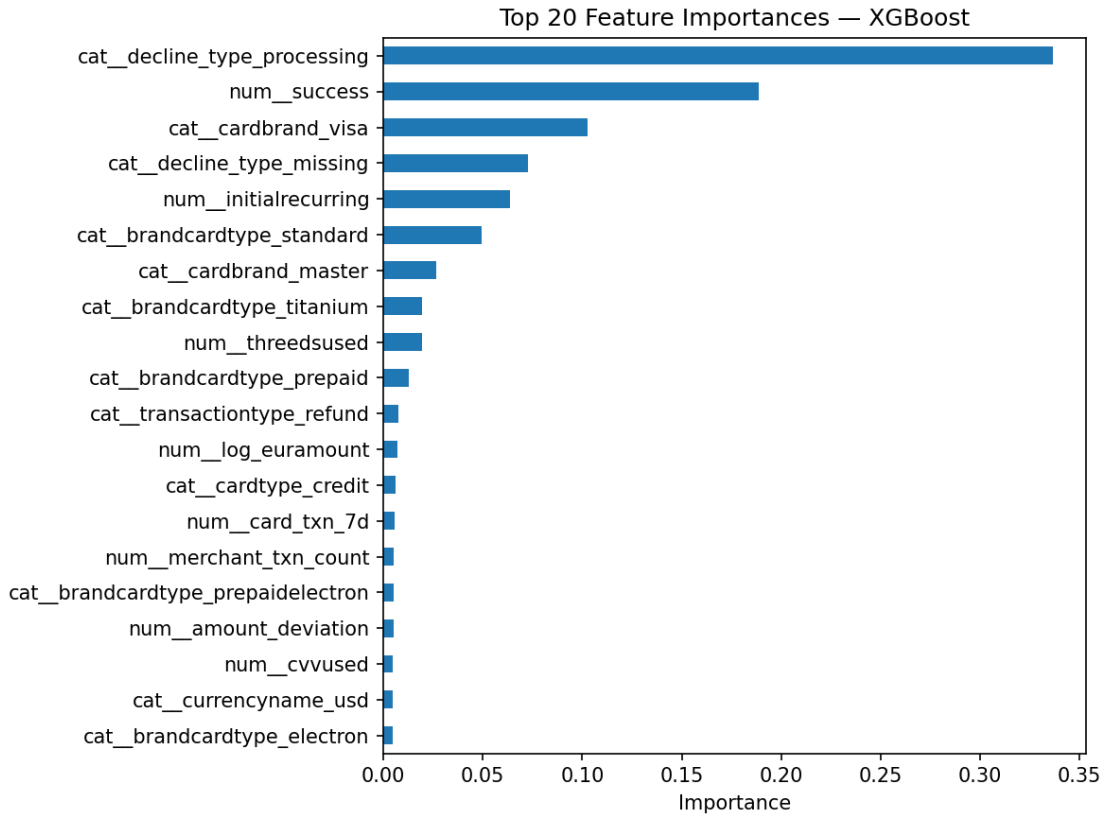
ROC Curve



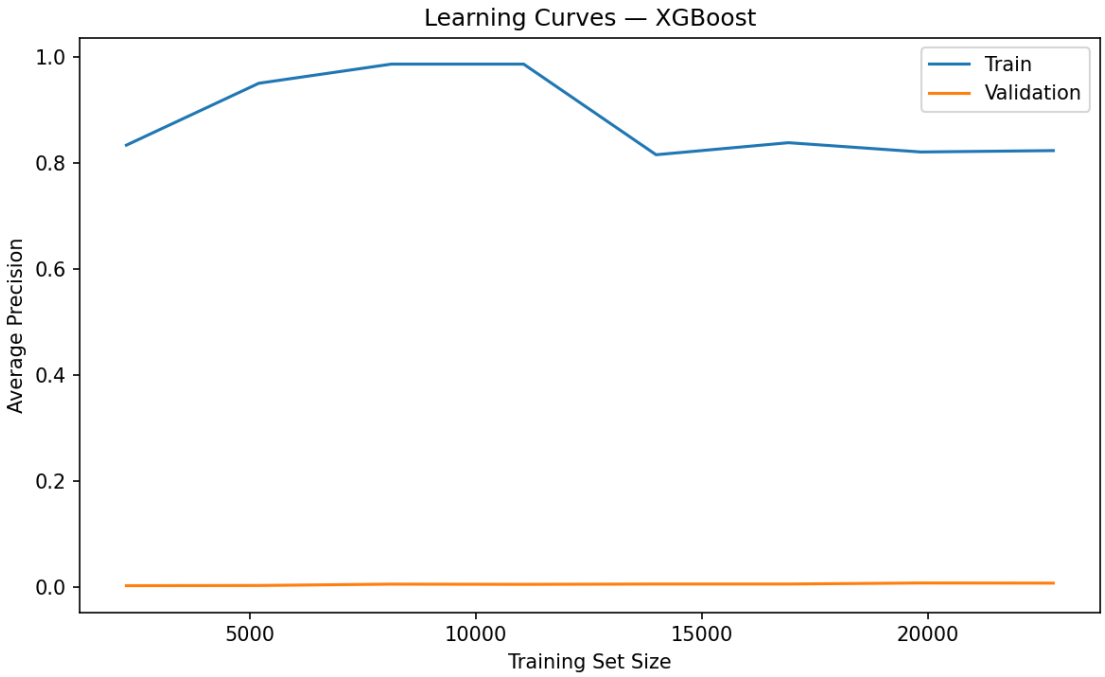
Precision-Recall Curve



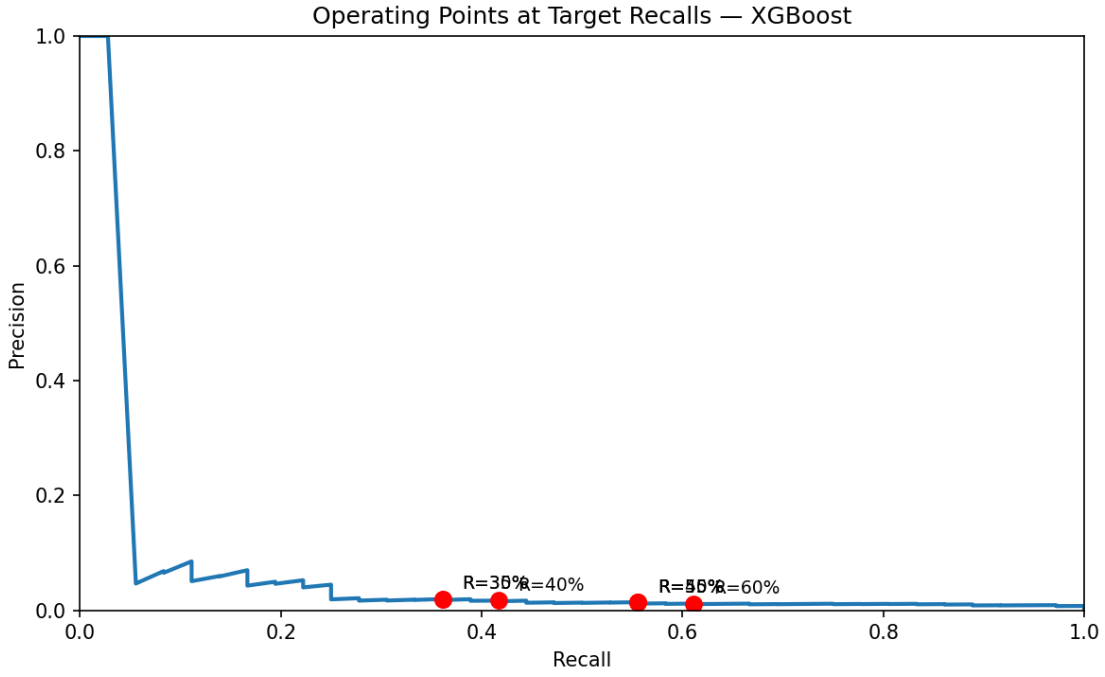
Feature Importance



Learning Curves

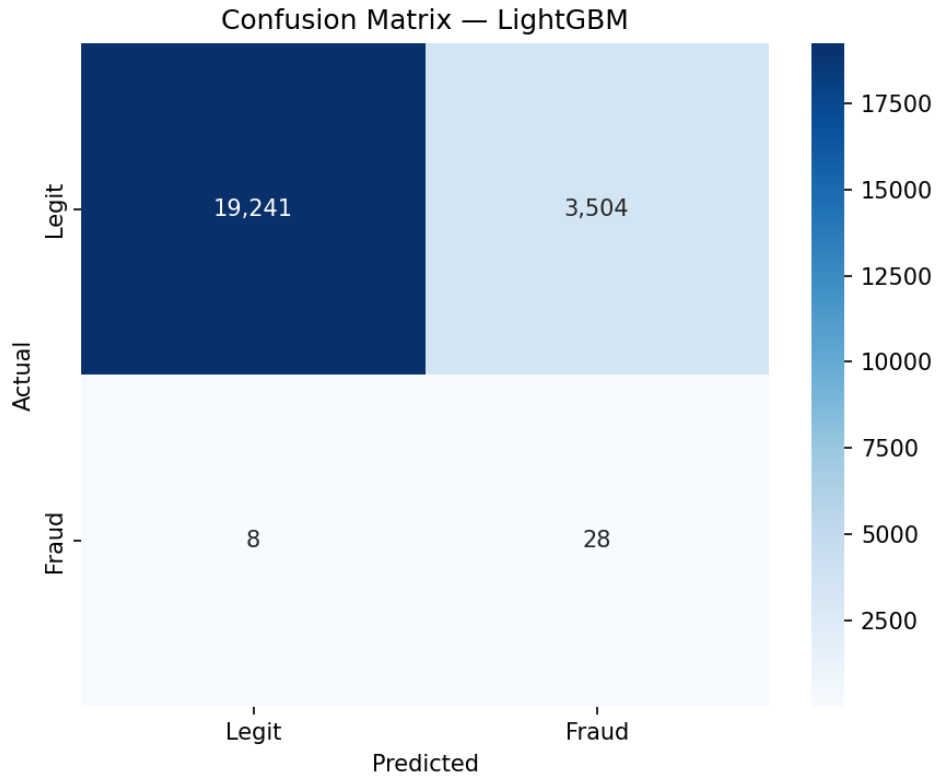


Recall Operating Points

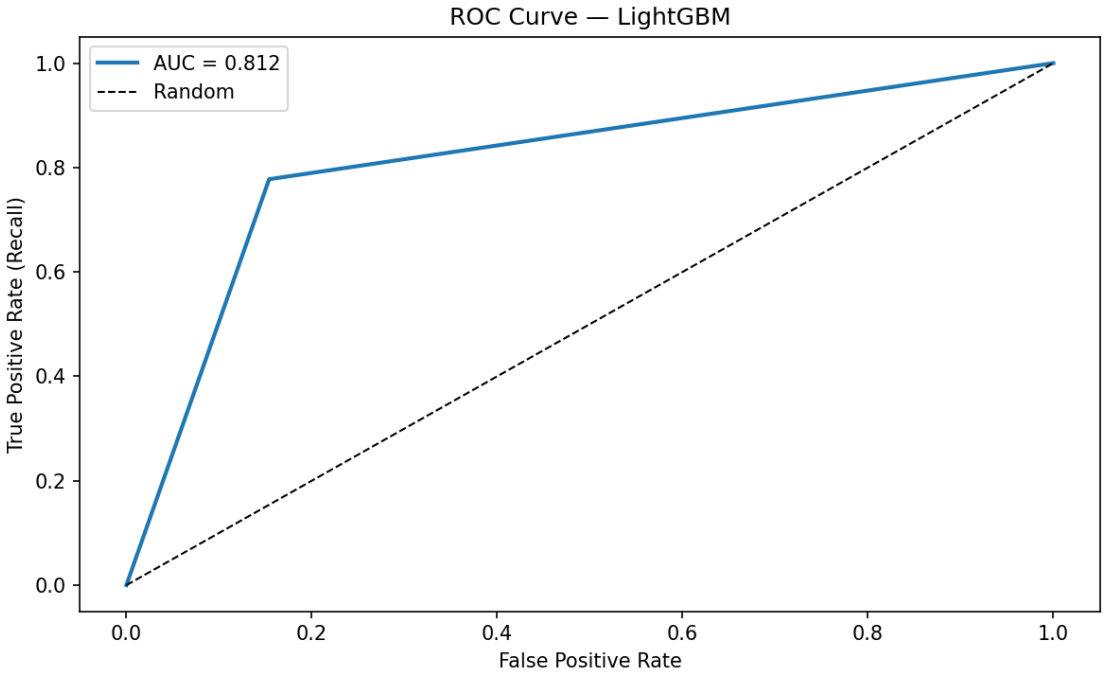


6.3 LightGBM -- Detailed Plots

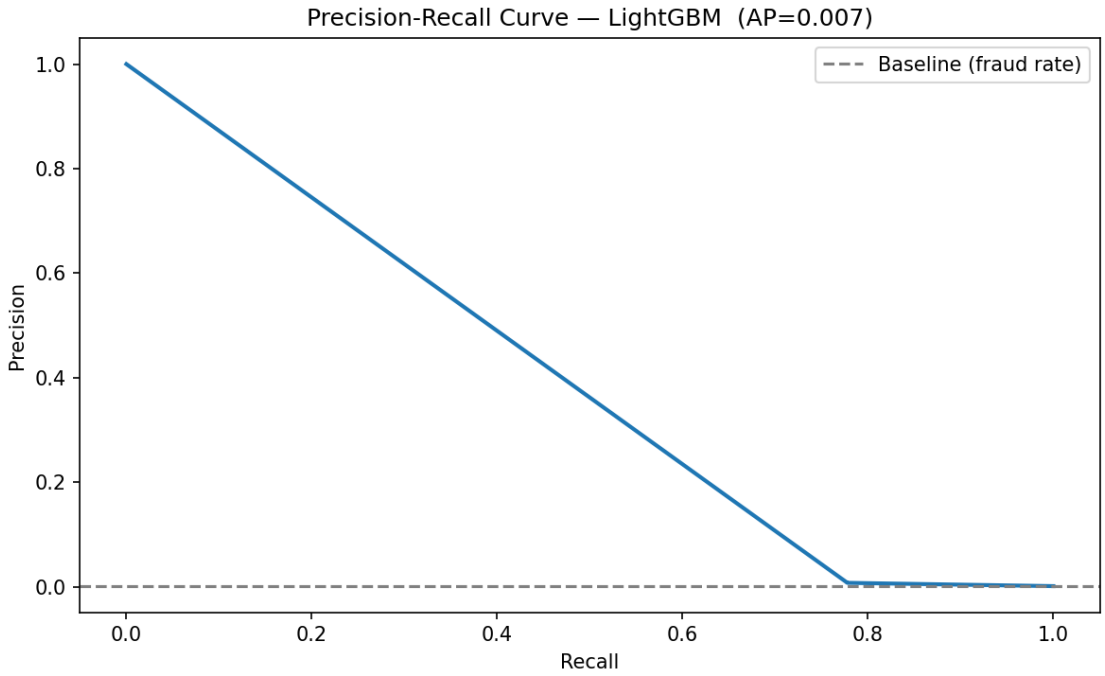
Confusion Matrix



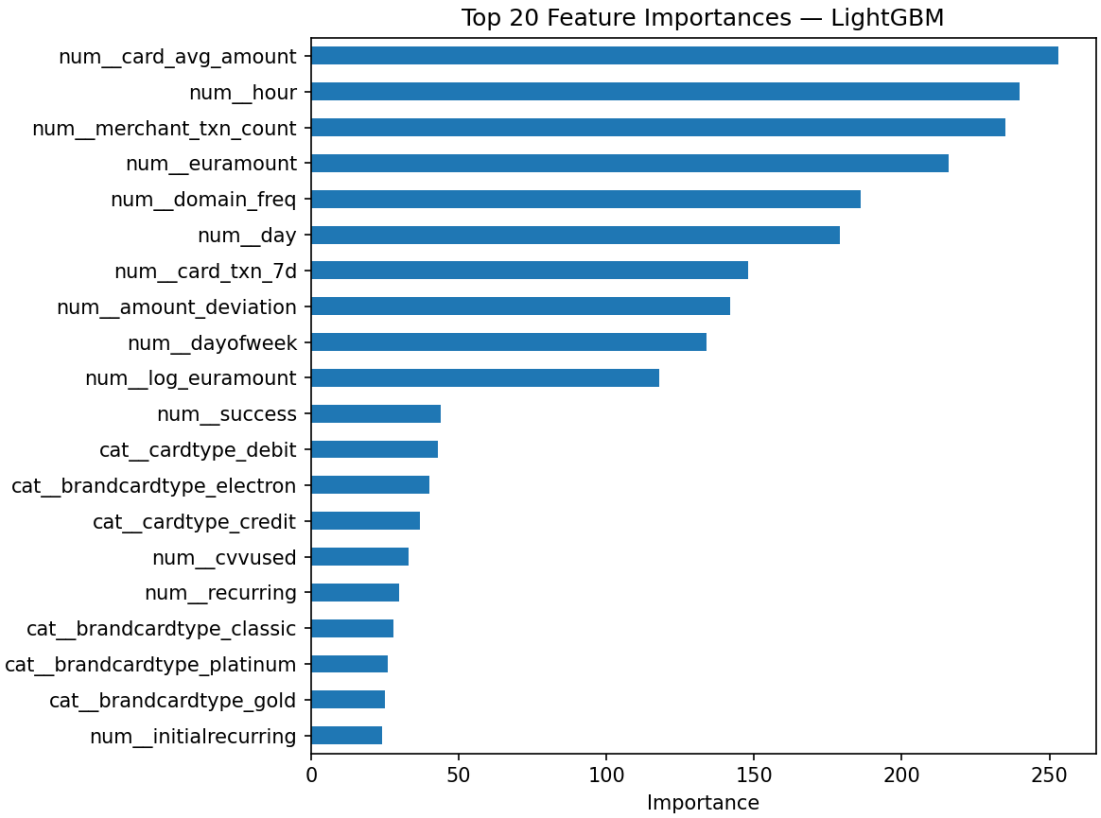
ROC Curve



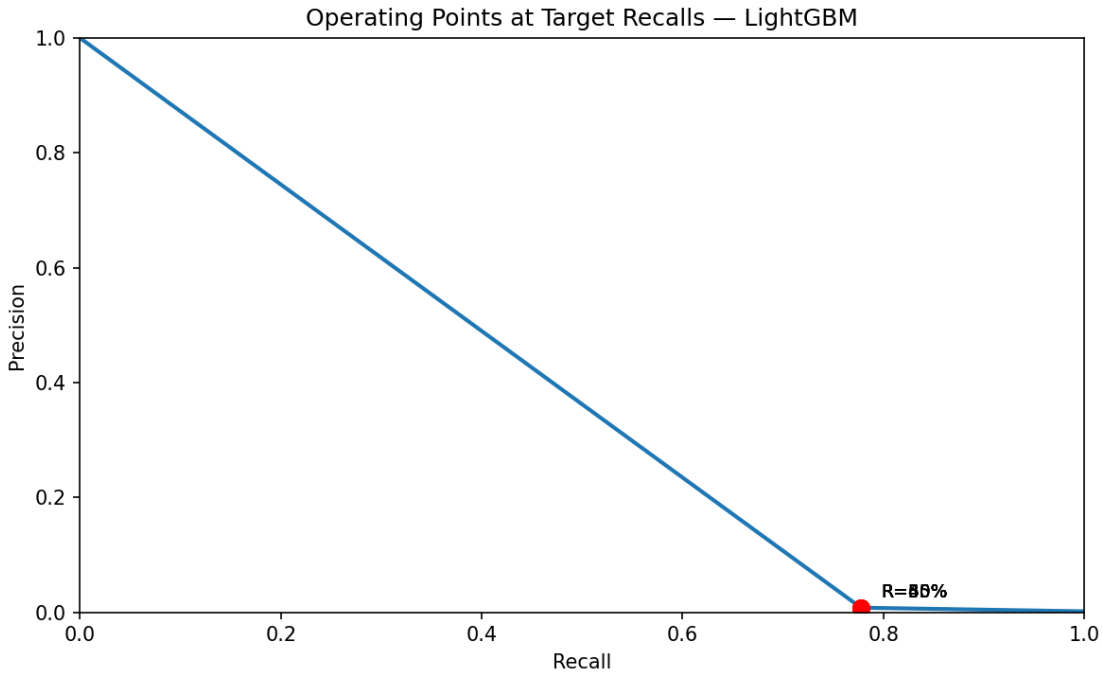
Precision-Recall Curve



Feature Importance



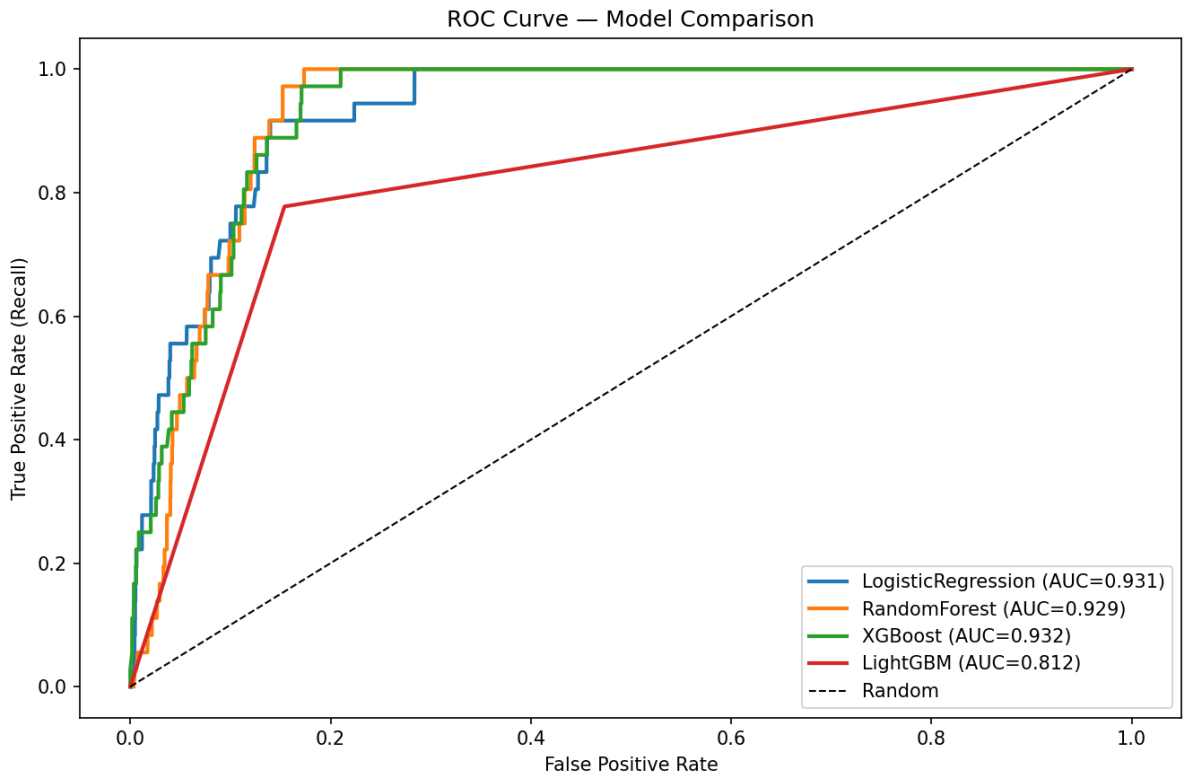
Recall Operating Points


Operating Points at Target Recalls — LightGBM
Precision
Recall
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

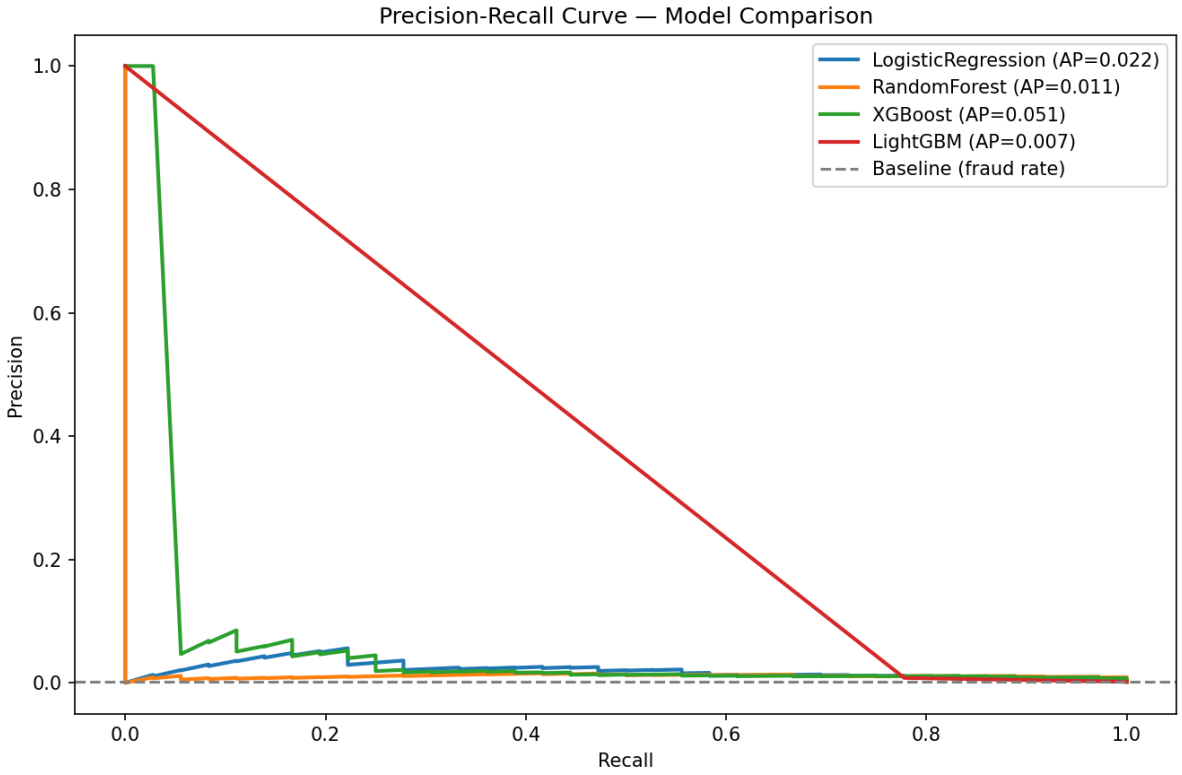
7. Model Comparison

The following plots compare all four models side-by-side on the validation set.

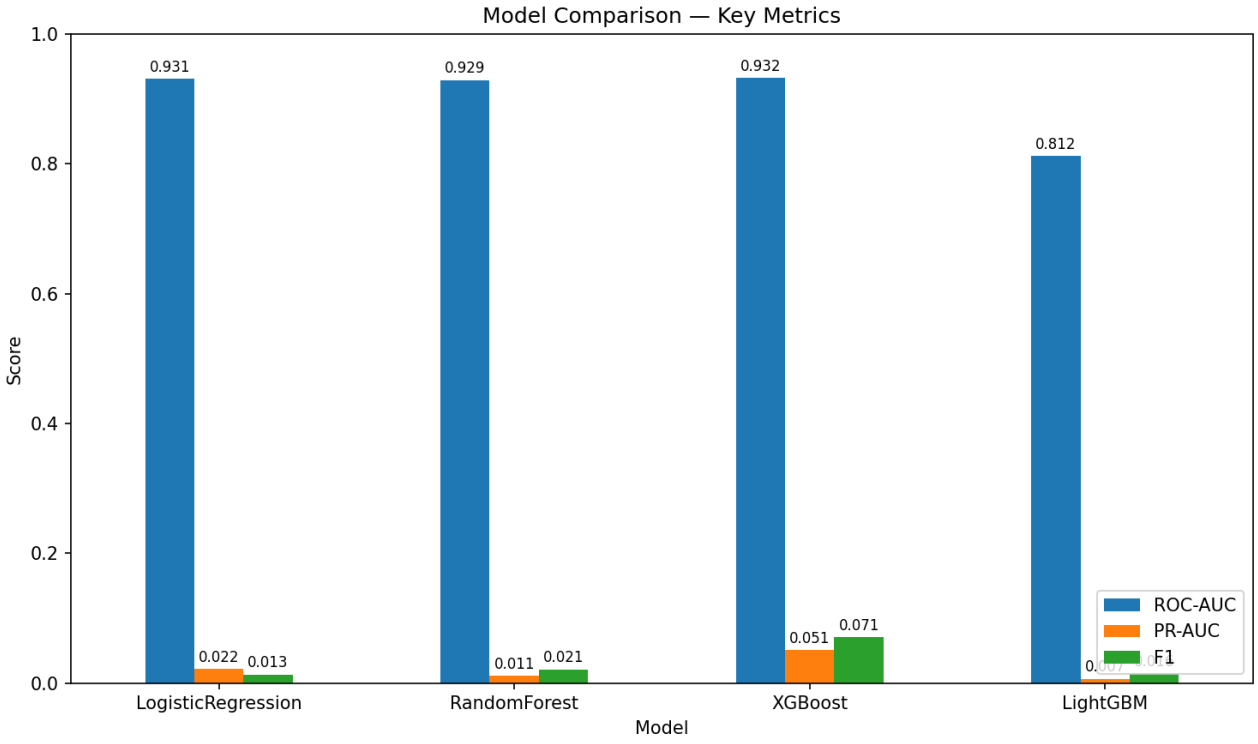
7.1 ROC Comparison



7.2 Precision-Recall Comparison



7.3 Metrics Bar Chart



Key observation: All models achieve high ROC-AUC (0.81-0.93), but PR-AUC values are very low (0.006-0.051), reflecting the extreme difficulty of achieving high precision on a 0.16% fraud-rate dataset. XGBoost leads by a meaningful margin in PR-AUC -- the metric most relevant for fraud detection deployment.

8. Recall Optimisation (30%–60%)

For each model, we compute the decision threshold that achieves target recall levels from 30% to 60%. This allows operational flexibility -- e.g. a higher recall catches more fraud but generates more false positives for manual review.

8.1 LogisticRegression

Target Recall	Threshold	Actual Recall	Precision	F1
30%	0.9362	41.67%	0.0259	0.0488
35%	0.9362	41.67%	0.0259	0.0488
40%	0.9362	41.67%	0.0259	0.0488
45%	0.9307	47.22%	0.0256	0.0486
50%	0.9048	55.56%	0.0216	0.0415
55%	0.9048	55.56%	0.0216	0.0415
60%	0.8549	69.44%	0.0134	0.0264

8.1 RandomForest

Target Recall	Threshold	Actual Recall	Precision	F1
30%	0.6920	41.67%	0.0154	0.0297
35%	0.6920	41.67%	0.0154	0.0297
40%	0.6920	41.67%	0.0154	0.0297
45%	0.6543	47.22%	0.0149	0.0289
50%	0.6303	50.00%	0.0138	0.0268
55%	0.5771	66.67%	0.0134	0.0262
60%	0.5771	66.67%	0.0134	0.0262

8.1 XGBoost

Target Recall	Threshold	Actual Recall	Precision	F1
30%	0.1162	36.11%	0.0195	0.0370
35%	0.1162	36.11%	0.0195	0.0370
40%	0.0742	41.67%	0.0168	0.0324
45%	0.0247	55.56%	0.0140	0.0273
50%	0.0247	55.56%	0.0140	0.0273
55%	0.0247	55.56%	0.0140	0.0273
60%	0.0106	61.11%	0.0116	0.0228

8.1 LightGBM

Target Recall	Threshold	Actual Recall	Precision	F1
30%	1.0000	77.78%	0.0079	0.0157
35%	1.0000	77.78%	0.0079	0.0157

40%	1.0000	77.78%	0.0079	0.0157
45%	1.0000	77.78%	0.0079	0.0157
50%	1.0000	77.78%	0.0079	0.0157
55%	1.0000	77.78%	0.0079	0.0157
60%	1.0000	77.78%	0.0079	0.0157

Observations:

- XGBoost achieves recall targets with much lower thresholds (0.01-0.12), indicating better-calibrated probability estimates for the positive class.

- Logistic R
predicted p

9. Test Predictions

Final test predictions are generated using the best model (XGBoost) at the 50% recall operating point. The threshold is calibrated on the validation set.

Property	Value
Best model	XGBoost
Decision threshold	0.0247
Target recall	~50%
Actual validation recall	55.56%
Validation precision	1.40%
Test transactions flagged	3,319 / 74,792 (4.44%)
Output file	test_predictions.csv

The test_predictions.csv file contains three columns: transactionid, fraud_probability (continuous score 0-1), and fraud_prediction (binary 0/1 using the chosen threshold).

10. Conclusions & Recommendations

10.1 Summary of Findings

- The pipeline correctly handles extreme class imbalance (0.11% fraud rate) through both algorithmic strategies and evaluation metric selection.
- Temporal s

10.2 Recommendations for Improvement

- Hyperparameter tuning: Run Optuna optimisation (TUNE_HYPERPARAMS=1) to reduce XGBoost overfitting while maintaining or improving PR-AUC.
- Early stopp
on 500 esti

10.3 Pipeline Architecture

The pipeline is modular and production-ready with clear separation of concerns:

- src/config.py -- Centralised configuration (features, paths, hyperparams)
- src/data_qu

Serialised artifacts (best_model.joblib, lookup_tables.joblib) enable deployment without re-training. The lookup tables ensure feature engineering at inference time uses only training-set statistics.